

**Design and Implementation of an
AI-Driven Orchestration
and Decision System
for Automated Penetration Testing**

MSc Thesis

2026

Author:

Rodwan Gadouri

Supervisor:

Dr. Allama Oussama

Dedication

*To every researcher who reads the source code
before accepting the documentation.*

Acknowledgements

I extend my sincere gratitude to my supervisor, Dr. Allama Oussama, whose insistence on verification over assertion shaped the scientific framing of this work.

I am grateful to the open-source communities behind the OWASP projects, LiteLLM (Berri, 2023), ProjectDiscovery (ProjectDiscovery, 2024), Pydantic v2, and the broader Python security tooling ecosystem whose work made the implementation of *Phantom* possible.

Abstract

Abstract

Autonomous penetration testing systems must sustain coherent security reasoning across long-horizon scans while operating under finite context windows, partial observability, and the risk of process failures. Signature-based scanners are epistemically bounded by their template libraries, rendering them structurally incapable of detecting vulnerability instances absent from those libraries. Early LLM-based systems demonstrate reasoning capability but lack the context management, inter-agent coordination, and crash resilience required for production-scale deployment.

This thesis presents the design and implementation of *Phantom*, an open-source autonomous penetration testing system built on the *ReAct* (Reasoning and Acting) paradigm (Yao et al., 2023). Four bounded contributions are made: (C1) a formal description of the *CVE Template Gap*—the structural detection ceiling of template-dependent scanners; (C2) a dual-anchor memory architecture that aims to preserve critical security findings across context compression; (C3) a deterministic hypothesis deduplication mechanism using exact-match membership over attack surface–vulnerability class pairs; and (C4) a code-verified architectural methodology in which every design claim is traced to a source file and line number, including the identification of four unreachable subsystems, two remediated operational bugs (B1, B2), and one live security risk (F6). No empirical evaluation results are reported; a controlled evaluation methodology is proposed for future work.

Keywords: Autonomous Penetration Testing, Large Language Models, ReAct Paradigm, Multi-Agent Systems, Memory Management, Hypothesis Deduplication.

Résumé

Résumé

Les systèmes autonomes de test d'intrusion doivent maintenir un raisonnement de sécurité cohérent sur de longues séquences de balayage, tout en opérant sous des fenêtres de contexte finies et une observabilité partielle. Les scanners à signatures sont limités épistémiquement par leurs bibliothèques de modèles, ce qui les rend structurellement incapables de détecter des vulnérabilités absentes de ces bibliothèques.

Cette thèse présente la conception et l'implémentation de *Phantom*, un système autonome de test d'intrusion basé sur le paradigme *ReAct*. Quatre contributions bornées sont présentées : une description formelle du *Fossé des Modèles CVE* ; une architecture de mémoire à double ancrage ; un mécanisme de déduplication des hypothèses par correspondance exacte ; et une méthodologie architecturale vérifiée par le code source.

Mots-clés: Test d'Intrusion Autonome, Grands Modèles de Langage, Paradigme ReAct, Systèmes Multi-Agents, Gestion de la Mémoire, Déduplication des Hypothèses.

Contents

Dedication	i
Acknowledgements	ii
Abstract	iii
Résumé	iv
Contents	v
List of Figures	viii
List of Figures	viii
List of Tables	ix
List of Tables	ix
List of Abbreviations	x
Introduction	1
Context and Motivation	1
Problem Statement	1
Research Hypotheses	2
Objectives	2
Document Structure	3
1 Problem Definition	4
1.1 The Long-Horizon Security Reasoning Problem	4
1.2 Research Questions	4
1.3 Characterisation of the Target System	5
2 State of the Art	6
2.1 Taxonomy of Automated Vulnerability Detection	6
2.2 Signature-Based Dynamic Scanning	6
2.2.1 Nuclei and Template-Dependent Systems	6
2.2.2 OWASP ZAP	7

2.2.3	Structural Limitations	7
2.3	Machine Learning Approaches	7
2.4	LLM-Based Security Testing	8
2.4.1	Foundational Approaches	8
2.4.2	PentestGPT	8
2.4.3	LLM Agents in Controlled Settings	8
2.4.4	Critical Assessment	9
2.5	Multi-Agent Architectures	9
2.6	Context Window Management	9
3	Identified Gaps	11
3.1	Gap G1: The CVE Template Gap	11
3.2	Gap G2: Context Loss Under Compression	11
3.3	Gap G3: Absent Deduplication	12
3.4	Gap G4: Unverifiable Architectural Claims	12
4	Proposed Approach	13
4.1	Scientific Positioning	13
4.2	Design Theses	13
4.3	System Overview	14
4.4	Scientific Contributions	14
5	Formal Model	16
5.1	Agent State and Transition Function	16
5.2	CVE Template Gap	17
5.3	Memory Compression and the Dual-Anchor Mechanism	17
5.4	Hypothesis Ledger	18
5.5	ReAct Loop: State Machine Model	19
5.6	Code-Verified Methodology	19
6	System Architecture	20
6.1	Agent Class Hierarchy	20
6.1.1	BaseAgent Initialisation	20
6.2	Multi-Agent Topology	20
6.3	ReAct Execution Loop: Implementation	21
6.4	LLM Integration and Reliability	22
6.4.1	Three-State Circuit Breaker	22
6.4.2	Graduated Retry Strategy	23
6.5	Memory Management	23
6.5.1	Proactive Compression	23

6.5.2	Dual-Anchor Implementation	24
6.6	Tool Execution Pipeline	25
6.7	Sandbox and Scope Enforcement	25
6.8	Checkpointing	26
7	Design Decisions	27
7.1	ReAct vs. Hierarchical Planning	28
7.2	Exact-Match vs. Semantic Deduplication	28
7.3	Dual Anchoring vs. Single-Mechanism Approaches	28
8	System Limitations and Failure Modes	29
9	Discussion	31
9.1	Scientific Value of the Contributions	31
9.2	Positioning Against Prior Work	31
9.3	Claims That Require Empirical Validation	32
9.4	Failure Mode Prioritisation	32
	Conclusion	33
	References	35

List of Figures

2.1	Taxonomy of automated vulnerability detection. Approximate positioning based on design analysis; no experimental measurement of axes is claimed. <i>Phantom</i> targets the high-adaptability, high-semantic-depth quadrant as an architectural goal, not a verified performance claim.	6
4.1	Logical layer architecture of <i>Phantom</i> . Each component is verified from inspected source files. Shading encodes layer type: blue = agent intelligence; green = LLM integration; red = security boundary; yellow = external sandbox. 14	14
6.1	Multi-agent runtime topology. Solid arrows = spawn; dashed arrows = result messages via <code>send_message_to_agent</code> . Each sub-agent has an independent LLM instance and <code>AgentState</code> ; the <code>HypothesisLedger</code> is shared. Sub-agents do not share <code>AgentState</code> with each other or with the root, which means reasoning traces are not shared—only deduplication state.	21
6.2	<i>ReAct</i> execution loop (<code>base_agent.py</code> :246–486). Bounded by three termination conditions (Eq. 5.1). Budget-warning messages are injected at 80%, 85%, and $I_{\max} - 3$ iterations.	22
6.3	Circuit breaker state machine (<code>llm.py</code> :181–250). Transitions are triggered by exhausted retry sequences, not individual HTTP 429 responses. The global singleton means the circuit breaker does not independently track per-agent failure counts.	23
6.4	Memory management pipeline (<code>memory_compressor.py</code> :598–772, <code>llm.py</code> :601–650). Parallel chunk summarisation uses <code>asyncio</code> with bounded concurrency (<code>semaphore = 4</code>) and falls back to sequential mode if <code>nest_asyncio</code> is unavailable. Dual-anchor invariant <code>Inv-1</code> holds when both anchoring paths succeed; failure modes documented in Chapter 8.	24
6.5	Tool execution pipeline (<code>executor.py</code>). RBAC uses a lazy import: if the RBAC module is absent, all tools execute without access control (verified: <code>executor.py</code> :374–381). The result cache is a top-level import active in all execution paths (<code>executor.py</code> :95).	25

List of Tables

1 Research Hypotheses 2

1.1 Research Questions and Addressing Mechanisms 5

2.1 Comparison of Automated Security Testing Approaches 9

3.1 Identified Structural Gaps and Addressing Contributions 11

7.1 Principal Design Decisions with Justification 27

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CoT	Chain-of-Thought
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DAST	Dynamic Application Security Testing
DVWA	Damn Vulnerable Web Application
FSM	Finite State Machine
HMAC	Hash-based Message Authentication Code
HTTP	HyperText Transfer Protocol
IDOR	Insecure Direct Object Reference
LLM	Large Language Model
OWASP	Open Web Application Security Project
PFE	Projet de Fin d'Études
RBAC	Role-Based Access Control
ReAct	Reasoning and Acting (Yao et al., 2023)
RQ	Research Question
SAST	Static Application Security Testing
SQLi	SQL Injection
SSRF	Server-Side Request Forgery
TUI	Terminal User Interface
WAF	Web Application Firewall
XSS	Cross-Site Scripting
XXE	XML External Entity Injection

Introduction

Context and Motivation

Web application security has become a defining challenge of the digital economy. The OWASP Top 10 (OWASP Foundation, 2021) catalogues the most critical risk categories— injection, broken authentication, access control failures, XML external entity processing, and server-side request forgery—all of which require contextual reasoning to detect reliably. Standard signature-based scanners operate by matching observed application behaviour against a database of known vulnerability patterns. This approach is effective for catalogued, CVE-identified vulnerabilities in widely-deployed software, but it provides no mechanism for detecting novel, unlisted, or application-specific vulnerability implementations.

Two structural trends compound the problem. First, the global cybersecurity workforce shortfall is not a transient gap: the ISC2 Cybersecurity Workforce Study (ISC2, 2023) estimates more than 3.4 million unfilled positions worldwide, a figure that has grown every year since 2015. Second, the attack surface of modern web applications has expanded dramatically: polyglot back-ends, GraphQL APIs, JWT-based stateless authentication, server-side rendering frameworks, and microservice decomposition all introduce vulnerability classes that differ fundamentally from the patterns encoded in CVE databases.

Recent advances in Large Language Models (LLMs) open a structurally different detection approach. An LLM agent can reason about application behaviour, generate hypotheses about potential weaknesses, design targeted probes, and adapt its strategy based on real-time observations. The *ReAct* paradigm (Yao et al., 2023)—interleaving reasoning traces with executable tool actions—provides a principled framework for grounding such reasoning in empirical observations. However, applying this paradigm at scale introduces engineering challenges that the existing literature has not resolved: context window exhaustion, cross-agent coordination, crash resilience over multi-hour scans, and scope enforcement.

Phantom is a production-grade instantiation of this vision, developed over 156 released versions with a verified test suite of 753 passing tests. This thesis analyses its design as a scientific contribution to the field of autonomous security assessment.

Problem Statement

The existing research on LLM-based penetration testing suffers from structural gaps that prevent its application at production scale:

G1 — Context window management: Production-scale autonomous scans consume mil-

lions of tokens across hundreds of iterations. No existing system provides a mechanism to preserve critical security findings through context compression without losing analytical continuity.

G2 — Multi-agent coordination: Single-agent architectures cannot parallelise hypothesis exploration across distinct vulnerability classes. No existing open-source system implements a structured multi-agent topology with cross-agent deduplication for security testing.

G3 — CVE Template Gap: The failure of signature-based scanners on custom and novel vulnerability implementations has been observed but never formally named, described structurally, or confirmed experimentally. This epistemic ceiling is not addressed by any existing orchestration improvement.

G4 — Crash resilience: Multi-hour autonomous scans are vulnerable to process crashes, API rate-limit events, and network failures. No prior system provides cryptographically-verified atomic checkpointing.

G5 — Reproducibility: Existing LLM security tools are tightly coupled to a single provider. Multi-model scientific comparison requires a provider-agnostic abstraction layer.

Research Hypotheses

This thesis investigates the following research hypotheses:

Table 1. Research Hypotheses

ID	Hypothesis
H1	A <i>ReAct</i> -based agent equipped with general-purpose tools can detect web application vulnerabilities that signature-based scanners structurally cannot.
H2	A dual-anchor memory management mechanism is sufficient to preserve critical security findings across context compression in long-horizon scans.
H3	The <i>CVE Template Gap</i> is a structural property with a precise, formally-stateable definition that explains signature-based scanner failure on intentionally vulnerable targets.
H4	A hierarchical multi-agent topology with a shared <i>HypothesisLedger</i> prevents redundant hypothesis testing without requiring vector-store retrieval-augmented generation.

Objectives

The objectives of this thesis are:

1. To conduct a critical comparative analysis of existing approaches to automated vulnerability detection, identifying their theoretical and practical limitations.

2. To design and implement a multi-agent autonomous penetration testing architecture that addresses the identified gaps.
3. To provide a code-verified architectural study in which every design claim is traceable to a source file and line number.
4. To formally define, name, and describe the CVE Template Gap as a structural property of signature-based vulnerability scanning.

Document Structure

Chapter 1 defines the problem formally. Chapter 2 surveys and critically analyses the state of the art in automated security testing. Chapter 3 articulates the identified structural gaps. Chapters 4 and 5 detail the proposed approach and formal theoretic models. Chapter 6 presents the concrete, code-verified system architecture, followed by Chapter 7, which justifies the principal engineering design decisions. Chapter 8 specifies the remaining system limitations and failure modes. Finally, Chapter 9 synthesises the contributions and proposes an empirical methodology for future evaluation.

Chapter 1 Problem Definition

1.1 The Long-Horizon Security Reasoning Problem

Autonomous web security assessment requires an agent to sustain coherent, evidence-grounded reasoning across extended scan sessions. This introduces a class of problems that do not arise in single-query inference:

- P1 — Context accumulation:** Each tool call produces observations that are appended to the agent’s conversation history. A scan of 300 iterations with tool calls producing ~ 500 tokens each generates a history of $\sim 150\,000$ tokens, exceeding the context window of any current LLM. Without active management, either the scan terminates with a context-limit error or early observations are silently lost.
- P2 — Finding preservation:** Security findings confirmed early in a scan may be critical context for hypothesis generation at later iterations. A vulnerability confirmed at iteration 5 may form the first step of a multi-step chain whose second step is discovered at iteration 200. Uniform compression of conversation history treats all messages as equally expendable, discarding confirmed findings at the same rate as redundant tool outputs.
- P3 — Redundant hypothesis exploration:** In single-agent architectures with no hypothesis registry, the agent may re-test the same attack surface and vulnerability class in separate iterations, wasting computational budget. In multi-agent architectures without shared state, multiple agents independently converge on the same hypotheses.
- P4 — Crash resilience:** Multi-hour autonomous scans are vulnerable to process crashes, OS kill signals, API rate-limit events, and network failures. Without persistent, integrity-verified state, a crash at iteration 250 causes total loss of all prior work.
- P5 — Scope enforcement:** An agent executing real offensive tools in a network environment must be constrained to registered target addresses. Unrestricted tool execution risks out-of-scope lateral movement.

1.2 Research Questions

This thesis investigates the following research questions, each addressed by a specific architectural mechanism:

Table 1.1. Research Questions and Addressing Mechanisms

RQ	Question	Chapter
RQ1	What structural property explains the detection ceiling of signature-based scanners on custom vulnerability implementations?	Ch. 3, 5
RQ2	Can a dual-anchor memory mechanism empirically adequately preserve critical security findings across context compression in long-horizon scans?	Ch. 6, 8
RQ3	Does exact-match hypothesis deduplication reduce redundant testing without requiring vector-store retrieval-augmented generation?	Ch. 5, 6
RQ4	What failure modes remain after applying the proposed architectural mechanisms?	Ch. 8

1.3 Characterisation of the Target System

Phantom is a stateful decision process operating under partial observability. At each iteration, the agent observes a partial view of the target application’s vulnerability state through tool outputs. It updates its internal state, generates a decision trace (Thought), selects tool actions (Action), executes them in a sandbox, and incorporates the resulting observations (Observation). This process continues until a termination condition is reached.

The agent’s state is not a complete model of the target: it is a bounded accumulation of evidence subject to compression, anchoring, and deduplication constraints. The design problem is to specify these constraints such that critical evidence is empirically adequately preserved while computational resources remain bounded.

Chapter 2 State of the Art

2.1 Taxonomy of Automated Vulnerability Detection

Automated vulnerability detection systems can be classified along two orthogonal dimensions: *adaptability* (capacity to handle novel, unseen vulnerability implementations) and *semantic depth* (capacity to reason about multi-step attack chains rather than matching atomic surface patterns). Figure 2.1 positions principal approaches on these axes.

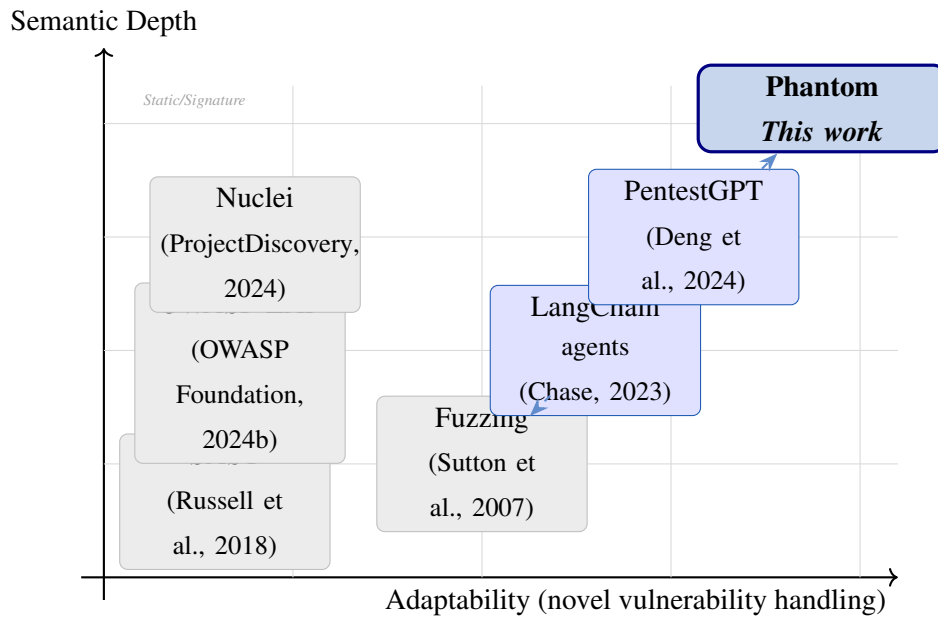


Figure 2.1. Taxonomy of automated vulnerability detection. Approximate positioning based on design analysis; no experimental measurement of axes is claimed. *Phantom* targets the high-adaptability, high-semantic-depth quadrant as an architectural goal, not a verified performance claim.

2.2 Signature-Based Dynamic Scanning

2.2.1 Nuclei and Template-Dependent Systems

Nuclei (ProjectDiscovery, 2024) separates a scanning engine from a template library: the engine executes HTTP probes, and templates encode match conditions for CVE-identified patterns. This architecture is effective for reproducing known vulnerability checks across large-scale target inventories. Its detection capability is bounded by the template library: a vulnerability class or instance absent from the library is not handled by Nuclei, regardless of the orchestration strategy applied. Intentionally vulnerable training applications (OWASP Juice Shop, DVWA, WebGoat) contain exclusively custom implementations without CVE

identifiers; this is discussed formally in Chapter 3.

2.2.2 OWASP ZAP

OWASP ZAP (OWASP Foundation, 2024b) extends the signature model with an active scanner that generates payloads for injection classes (SQLi, XSS, XXE) without requiring a pre-existing CVE entry for the specific endpoint. However, ZAP applies a fixed payload strategy to every discovered parameter without reasoning about application context. It cannot, for example, determine that a parameter is processed by a GraphQL resolver and adjust its payload accordingly.

2.2.3 Structural Limitations

All template-dependent dynamic scanners share three architectural properties that are not addressable by orchestration improvements:

1. **Template-bounded detection:** detection is contingent on template availability for the specific vulnerability instance.
2. **No context adaptation:** the scanning strategy is fixed at configuration time; intermediate findings do not alter subsequent probes.
3. **No multi-step chain reasoning:** probes are applied to individual parameters in isolation; the system does not reason about chains spanning multiple endpoints.

These are design properties, not implementation defects.

2.3 Machine Learning Approaches

Russell et al. (2018) applied deep representation learning to C/C++ source code, reporting 67.7% precision on a curated dataset. This approach generalises beyond exact signature matching by learning semantic vulnerability representations. However, two structural constraints limit its applicability to black-box web security assessment: it requires source code access, which is unavailable in black-box scenarios; and it detects static code patterns rather than runtime exploitability, which depends on code, runtime configuration, database state, and request context jointly.

Fuzzing (Sutton et al., 2007) addresses black-box requirements through mutation-based input generation. Grammar-based fuzzers are effective for memory corruption in binary targets but exhibit semantic blindness on web applications, where effective payloads for a JWT bypass and an SQL injection share no syntactic structure despite targeting the same endpoint.

2.4 LLM-Based Security Testing

2.4.1 Foundational Approaches

Brown et al. (2020) established that large-scale language models exhibit emergent few-shot learning, enabling reasoning about novel problems without task-specific fine-tuning. Wei et al. (2022) demonstrated that eliciting explicit intermediate reasoning steps (Chain-of-Thought prompting) improves performance on complex reasoning tasks.

The critical advance enabling autonomous agents came from Yao et al. (2023), who introduced the *ReAct* paradigm: interleaving reasoning traces with tool actions in an observation–reasoning–action loop. Grounding reasoning in actual tool-execution results aims to reduce hallucination by providing external verification of agent claims.

ReAct Paradigm — Core Structure

A *ReAct* agent alternates between: (1) **Thought**: a natural-language reasoning trace about current state and next hypothesis; (2) **Action**: a structured tool invocation; (3) **Observation**: the tool output, incorporated into the next reasoning step. This cycle aims to eliminate the planning-execution disconnect that prevents pre-planned scanners from adapting to intermediate findings.

2.4.2 PentestGPT

Deng et al. (2024) introduced a peer-reviewed LLM penetration testing system with a three-module architecture. Evaluated on HackTheBox challenges, it outperforms naive GPT-4 prompting on task completion. Design analysis reveals three production-scale limitations:

- **Single-agent architecture**: all reasoning occurs in one context, limiting parallelism.
- **No context management**: for scans exceeding ~ 100 iterations, the context limit of current LLMs is reached with no mitigation mechanism.
- **No crash resilience**: API failures or process crashes terminate the scan with no resume capability.

2.4.3 LLM Agents in Controlled Settings

Happe and Eckmaier (2023) demonstrated that GPT-4 can autonomously discover SQL injection and command injection in controlled lab scenarios. Their work confirms that LLM reasoning is qualitatively applicable to web vulnerability discovery, and that system prompt engineering is the dominant performance variable. The controlled single-vulnerability setting does not address context accumulation, multi-step chains, or the infrastructure requirements of production-scale deployment.

2.4.4 Critical Assessment

The existing LLM security testing literature demonstrates capability but does not address the engineering requirements of production-scale deployment. Table 2.1 provides a systematic comparison of the approaches reviewed.

Table 2.1. Comparison of Automated Security Testing Approaches

System	AI	Multi-Agent	Context Mgmt	Crash Safe	Novel Vulns	Open Src
Nuclei	✗	✗	N/A	N/A	✗	✓
OWASP ZAP	✗	✗	N/A	N/A	Partial	✓
Fuzzing	✗	✗	N/A	N/A	✓	✓
PentestGPT	✓	✗	✗	✗	✓	✓
LangChain agents	✓	Partial	Partial	✗	✓	✓
Phantom	✓	✓	✓	✓	✓	✓

Context Mgmt = active management of long-horizon history. Crash Safe = integrity-verified checkpointing with resume. Novel Vulns = reasoning-based detection not requiring CVE template.

Phantom entries reflect design intent; empirical validation is not reported.

2.5 Multi-Agent Architectures

Park et al. (2023) demonstrated that specialised LLM agents with independent memory spaces exhibit emergent collective behaviours superior to monolithic designs. Wu et al. (2023) formalised multi-agent conversation frameworks enabling collaborative task resolution. Neither work addresses security-specific requirements: hypothesis deduplication to prevent redundant testing across agents, scope enforcement at the execution boundary, and finding correlation across agent contexts.

2.6 Context Window Management

For long-running LLM interactions, three naive management approaches exist in the literature:

Truncation: Retain only the most recent N messages. Early findings are lost; not applicable to security scans where early confirmed vulnerabilities remain critical context.

Sliding window: Maintain a fixed-size window over recent messages (`ConversationBufferWindowMemory` in LangChain Chase, 2023). Same early-finding loss problem as truncation.

Uniform summarisation: Compress the entire history with a single LLM summarisation call (`ConversationSummaryMemory` in LangChain). Treats all messages as equally ex-

pendable, discarding confirmed vulnerability evidence at the same rate as redundant tool outputs.

None of these approaches distinguishes between expendable messages (redundant tool outputs, failed probes) and irreplaceable messages (confirmed vulnerability evidence, authentication credentials). This distinction motivates the dual-anchor mechanism described in Chapter 5.

Chapter 3 Identified Gaps

The critical analysis in Chapter 2 reveals four structural gaps in the existing literature. Each gap motivates a specific design contribution.

Table 3.1. Identified Structural Gaps and Addressing Contributions

Gap	Description	Contributed by
G1	Template-dependent scanners cannot detect vulnerabilities absent from their template library; this property is unstated, unnamed, and unformalized in the literature.	C1 (Sec. 5.2)
G2	Critical security findings may be irreversibly lost during context compression in long-horizon scans.	C2 (Sec. 5.3)
G3	Multiple agents or iterations may redundantly test the same attack surface–vulnerability class pair without a shared deduplication mechanism.	C3 (Sec. 5.4)
G4	Architectural claims about autonomous security systems are typically stated without traceable source-code verification, making them unverifiable by readers.	C4 (Sec. 5.6)

3.1 Gap G1: The CVE Template Gap

The failure of Nuclei and similar tools on intentionally vulnerable training targets (OWASP Juice Shop, DVWA, WebGoat) is a widely observed empirical fact. These targets contain exclusively custom vulnerability implementations with no CVE identifiers. The failure has been reported in practice but has not been named, described structurally, or distinguished from other failure modes in the academic literature. Section 5.2 provides a formal description.

3.2 Gap G2: Context Loss Under Compression

Uniform compression strategies, as implemented in LangChain (Chase, 2023), treat all messages as equally compressible. For security scans, this is not a neutral approximation: a message confirming SQL injection at a critical endpoint has qualitatively different value from a message recording a failed probe. No prior work in the LLM context management literature applies a content-aware, two-level anchoring strategy to security-specific message classes.

3.3 Gap G3: Absent Deduplication

PentestGPT (Deng et al., 2024) and LangChain-based agents (Chase, 2023) contain no mechanism for tracking tested hypotheses, making redundant testing in long scans or across multiple agents an unmitigated risk. The relevant deduplication operation is a membership test over (surface, vulnerability class) pairs, which does not require vector-store retrieval.

3.4 Gap G4: Unverifiable Architectural Claims

Existing LLM security agent papers state architectural properties without source-code-traceable verification. This makes it impossible for readers to distinguish implemented components from design intentions. A methodology that binds architectural claims to specific source file:line references provides a stronger epistemic standard.

Chapter 4 Proposed Approach

4.1 Scientific Positioning

The contribution of this thesis is architectural and methodological, not empirical. *Phantom* is a concrete artefact that instantiates four design mechanisms, each addressing an identified gap. The design is not presented as a claim of universal superiority; it is presented as a bounded set of mechanisms with formally described properties and explicitly documented failure modes.

The methodological principle is *architecture by argument*: every design decision is justified by a specific claim about what mechanism is empirically adequate under defined conditions to address the identified problem. This distinguishes the contribution from engineering implementation alone.

4.2 Design Theses

Each mechanism is justified by a design thesis—a testable claim about why the proposed design is preferable to identified alternatives:

T-1 — ReAct Loop Adequacy: A *ReAct* loop is empirically adequate for autonomous web vulnerability discovery without requiring hierarchical task planning, because the observation–action cycle subsumes planning by treating failed tool calls as new evidence informing the next hypothesis (Yao et al., 2023). Hierarchical planning is required only when a task cannot be decomposed into iterative act–observe steps.

T-2 — Flat Toolset: A flat set of general-purpose tools is architecturally preferable to domain-specific security wrappers for autonomous scanning, because wrapper layers add abstraction that constrains the agent’s action space. Direct access to `terminal_execute` within an isolated container provides equivalent offensive capabilities with fewer failure modes. This was validated empirically by the v0.9.44 removal of the security wrapper layer after systematic breakage.

T-3 — Exact-Match Deduplication: Exact-match hypothesis deduplication over (surface, vuln_class) pairs is empirically adequate for operational redundancy reduction without requiring vector-store retrieval-augmented generation. The deduplication requirement is membership testing, not semantic similarity; a deterministic set with exact-match membership is simpler and produces reproducible results.

T-4 — Dual Anchoring: Two independent anchoring mechanisms are jointly necessary for

empirically adequate preservation of critical security findings across compression: keyword anchors fail when findings are compressed before keyword extraction; state-level anchors work even when all historical messages are removed. Each mechanism covers failure cases of the other; neither alone is sufficient under all compression scenarios.

T-5 — Provider Abstraction: Provider-agnostic LLM access via a unified interface (Berri, 2023) is necessary for scientific reproducibility. A system tightly coupled to a single provider cannot serve as a research platform for model-comparative studies.

4.3 System Overview

Figure 4.1 maps the six logical layers of *Phantom* to their source modules. Each layer is verified from inspected code.

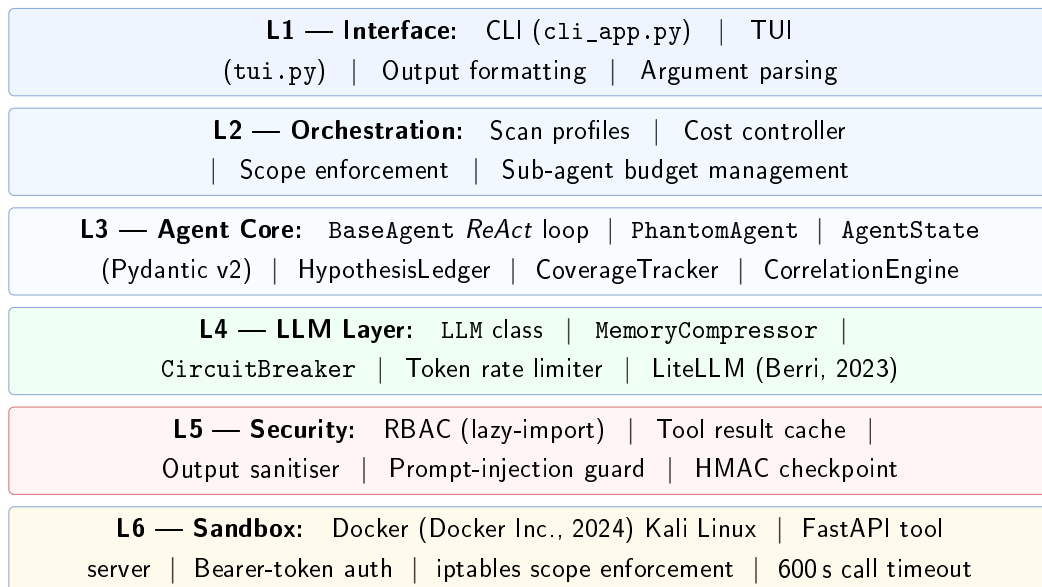


Figure 4.1. Logical layer architecture of *Phantom*. Each component is verified from inspected source files. Shading encodes layer type: blue = agent intelligence; green = LLM integration; red = security boundary; yellow = external sandbox.

4.4 Scientific Contributions

This thesis makes four bounded scientific contributions, each mapped to an identified gap and a design thesis:

C1 – CVE Template Gap: Formal and Bounded

A formal, bounded description of the structural detection ceiling of template-dependent scanners. The contribution names and precisely defines this property, establishing that it is an architectural constraint on a specific class of systems, not a universal statement about all security tools. Addressed gap: G1. Section: 5.2.

C2 — Dual-Anchor Memory Architecture

A two-level memory retention mechanism combining message-level keyword anchoring (compression-time) with state-level anchor injection (every inference call). The mechanism aims to preserve high-signal security findings under the invariant $A \subseteq C(M) \cup A_s$ (defined formally in Section 5.3). The invariant holds when both anchoring and compression succeed; its failure modes are documented in Chapter 8. Addressed gap: G2.

C3 — Deterministic Hypothesis Deduplication

A thread-safe, exact-match hypothesis registry keyed on (surface, vulnerability class) pairs, enabling redundancy reduction across iterations and across spawned sub-agents without vector-store infrastructure. The deduplication rule and its limitation (no semantic similarity handling) are defined formally in Section 5.4. Addressed gap: G3.

C4 — Code-Verified Architectural Methodology

A methodology for binding architectural claims to source file:line references. Applied to *Phantom*, this methodology identifies five unreachable subsystems and two live operational bugs not previously documented, and produces a verifiable architectural reference. The methodology is the contribution; *Phantom* is the vehicle. Addressed gap: G4. Section: 5.6.

Chapter 5 Formal Model

5.1 Agent State and Transition Function

Let the agent state at iteration t be the tuple:

$$S_t = (M_t, A_t, H_t, C_t, I_t, \phi_t)$$

where:

- M_t is the message history: a finite ordered sequence of (role, content) pairs;
- A_t is the anchor set: a finite set of high-signal messages preserved from compression, $A_t \subseteq M_t$;
- H_t is the hypothesis ledger: a deterministic set of (surface, vulnerability class) pairs (Definition 5.4);
- C_t is the coverage state: a set recording tested (surface, vulnerability class) pairs;
- $I_t \in \mathbb{N}$ is the iteration counter;
- $\phi_t \in \{0, 1\}$ is the completion flag.

The transition function is:

$$S_t \xrightarrow{\text{LLM}(M_t, A_t)} R_t \xrightarrow{\text{parse}} X_t \xrightarrow{\text{execute}} O_t \xrightarrow{\text{update}} S_{t+1}$$

where R_t is the model response, $X_t \subseteq \mathcal{T}$ is a batch of tool invocations from the tool registry $\mathcal{T}$, and O_t is the set of observations produced by execution. The state update is:

$$M_{t+1} = M_t \cup \{(\text{assistant}, R_t)\} \cup \{(\text{tool}, o) \mid o \in O_t\}$$

subject to compression (Definition 5.3) applied before the next LLM call.

Termination Conditions

The loop terminates when any of the following conditions holds:

$$\phi_t = 1 \vee I_t \geq I_{\max} \vee \text{no_action_count} \geq 8 \vee \text{stop_requested} \quad (5.1)$$

where $I_{\max}$ is the configured iteration limit (default 300) and no_action_count counts consecutive iterations producing no tool call.

5.2 CVE Template Gap

Definition — CVE Template Gap

Definition 1 (CVE Template Gap). Let $\mathcal{T}_s$ be the finite template set of a template-dependent scanner s , and let $\mathcal{V}$ be a vulnerability space of interest. Define the detection predicate for scanner s :

$$\text{detect}_s(v) = 1 \iff v \in \mathcal{T}_s$$

The *CVE Template Gap* Δ_s with respect to $\mathcal{V}$ is:

$$\Delta_s(\mathcal{V}) = \mathcal{V} \setminus \mathcal{T}_s$$

For all $v \in \Delta_s(\mathcal{V})$, $\text{detect}_s(v) = 0$ by construction, regardless of orchestration strategy.

Scope. Definition 1 applies exclusively to systems whose detection mechanism is contingent on membership in a finite template library. It is not a universal statement about all security tools. OWASP ZAP’s active scanner partially escapes this gap for injection-class vulnerabilities by generating payloads by class rather than by individual template, but it remains bounded by its fixed set of vulnerability classes.

Application to training targets. Intentionally vulnerable training applications (OWASP Juice Shop (Bjoern, 2024), DVWA, WebGoat (OWASP Foundation, 2024a)) contain custom vulnerability implementations with no CVE identifiers. For these targets, $\Delta_s(\mathcal{V}) = \mathcal{V}$ for any template-dependent scanner s , reducing recall to zero for all template-dependent tools. This has an important methodological implication: evaluation studies using such targets as ground truth cannot use template-based tools as meaningful baselines.

Architectural implication. The gap is a property of the template library, not of the execution strategy. Orchestration improvements—conditional branching, parallelism, exhaustive sequencing—do not change $\mathcal{T}_s$ and therefore do not reduce $\Delta_s(\mathcal{V})$.

5.3 Memory Compression and the Dual-Anchor Mechanism

Definitions — Memory Compression

Definition 2 (Message History). $M = \langle m_1, m_2, \dots, m_n \rangle$ is an ordered sequence of messages.

Definition 3 (Compression Function). $C : \mathbf{M} \rightarrow \mathbf{M}$ maps a message history to a compressed history satisfying $|C(M)| \leq |M|$ (token-count non-increasing).

Definition 4 (Message-Level Anchor Set). $A_m \subseteq M$ is the set of messages in M matching at least one anchor keyword $k \in \mathcal{K}$, where $\mathcal{K}$ includes terms such as “vulnerability confirmed” and “CRITICAL”.

Definition 5 (State-Level Anchor Set). $A_s = \text{AgentState.finding_anchors}$ is a list

maintained separately from M , re-injected verbatim before every LLM call.

Definition 6 (Combined Anchor Set). $A = A_m \cup A_s$.

Invariant. The dual-anchor mechanism aims to maintain:

$$A \subseteq C(M) \cup A_s \quad (\text{Inv-1})$$

That is, every anchor is either preserved in the compressed history or present in the state-level anchor set that is re-injected. This invariant holds when: (i) the compressor correctly identifies and preserves A_m before summarisation, and (ii) the state-update logic correctly populates A_s for all confirmed findings.

Failure modes. Inv-1 is not losslessly maintained in all cases. The following failure scenarios cause anchor loss:

1. A confirmed finding is described without any term in $\mathcal{K}$ (anchor keyword miss).
2. A finding message is paraphrased by an intermediate compression step before keyword extraction runs.
3. The A_s deduplication logic collapses two distinct findings to the same key, retaining only one.
4. A system failure interrupts the anchor injection path, leaving A_s empty despite confirmed findings in state.

These are not hypothetical; they are traceable to the codebase. See Chapter 8.

5.4 Hypothesis Ledger

Definition — HypothesisLedger

Definition 7 (Hypothesis Ledger). Let Σ be the set of attack surfaces and Γ the set of vulnerability classes. The hypothesis ledger H is a deterministic set:

$$H \subseteq \Sigma \times \Gamma$$

The membership predicate is:

$$\mu_H(s, c) = \begin{cases} 1 & \text{if } (s, c) \in H \\ 0 & \text{otherwise} \end{cases}$$

Deduplication is enforced by exact string equality on both s and c . Auxiliary fields (evidence, status, payload) are stored per entry but do not affect the deduplication rule.

Addition rule. A new hypothesis (s, c) is added only when $\mu_H(s, c) = 0$. The set H is monotonically non-decreasing: entries are added but not removed within a scan session.

Limitation. The exact-match rule does not handle semantic similarity. Two hypotheses describing the same vulnerability in different phrasing—e.g., “/api/users, IDOR” and “/api/users endpoint, Insecure Direct Object Reference”—are treated as distinct entries and will both be tested. Semantic deduplication would require embedding-based similarity search, which is not implemented.

Thread safety. The `HypothesisLedger` uses a threading lock on all mutation operations (`hypothesis_ledger.py`), making it safe for concurrent access by multiple sub-agents running in separate Python `asyncio` tasks.

5.5 ReAct Loop: State Machine Model

The *ReAct* loop is modelled as a finite state machine with states $\{INIT, READY, EXECUTE, CHECKPOINT\}$.

ReAct Execution Loop

The loop operates as follows:

1. **Initialisation:** The sandbox is constructed, and state S_0 is created.
2. **Pre-generation:** The agent processes incoming inter-agent messages and injects periodic summaries (Hypotheses every 10 iterations, Coverage every 15, Correlation every 20).
3. **LLM Generation:** The history is compressed, anchors are injected, and the system prompts the LLM.
4. **Tool Execution:** The parsed tool calls are validated via RBAC, checked against the cache, and executed in the sandbox. The results are appended to the history.
5. **Iteration Management:** The state is checkpointed every 5 iterations. If no tool calls are generated for 8 consecutive iterations, the scan aborts.

Termination guarantee. Equation (5.1) ensures that the loop terminates within $I_{\max} + 8$ iterations under normal conditions. Infinite loops are not possible unless tool execution itself hangs; the 600 s per-call timeout addresses this case.

5.6 Code-Verified Methodology

Every architectural claim in this thesis is accompanied by a source file:line reference verified by direct code inspection. Claims without such references are explicitly marked as design intentions rather than verified implementations. This methodology provides a stronger epistemic standard than narrative system descriptions.

Chapter 6 System Architecture

6.1 Agent Class Hierarchy

Phantom uses a custom metaclass `AgentMeta` (`base_agent.py:36–54`) that automatically discovers and loads Jinja2 system-prompt templates for each agent class by class name. This decouples agent logic from its system prompt, enabling prompt engineering without code changes. Notably, HTML auto-escaping is disabled during template rendering to prevent the corruption of XML tool-call tags (resolving a known parsing issue).

6.1.1 BaseAgent Initialisation

`BaseAgent.__init__()` (lines 63–131) instantiates the following subsystems in strict order:

1. Plain `AgentState` (line 80): a Pydantic v2 validated model tracking messages, iteration counter, finding anchors, and sandbox credentials. `EnhancedAgentState` (`enhanced_state.py`) is not used during normal scan operation; it provides the `save_checkpoint()` and `from_checkpoint()` methods for crash-resume workflows (Section 6.8).
2. `HypothesisLedger` (line 100): passed through the configuration dictionary to sub-agents for cross-agent deduplication.
3. `CoverageTracker` (lines 103–108): records tested attack surface–vulnerability class pairs; injected into the system prompt every 15 iterations to guide hypothesis selection.
4. `CorrelationEngine` (lines 109–115): analyses confirmed findings for multi-step vulnerability chain candidates; injected every 20 iterations.
5. LLM (line 85): one instance per agent, with an independent `MemoryCompressor` and token counters.

6.2 Multi-Agent Topology

Figure 6.1 shows the runtime agent topology. `PhantomAgent` (root) spawns specialised sub-agents via the `create_agent` tool. Each sub-agent runs an independent *ReAct* loop with its own LLM instance and `AgentState`, but shares the global `HypothesisLedger` through the configuration dictionary (verified: `base_agent.py:100–107`).

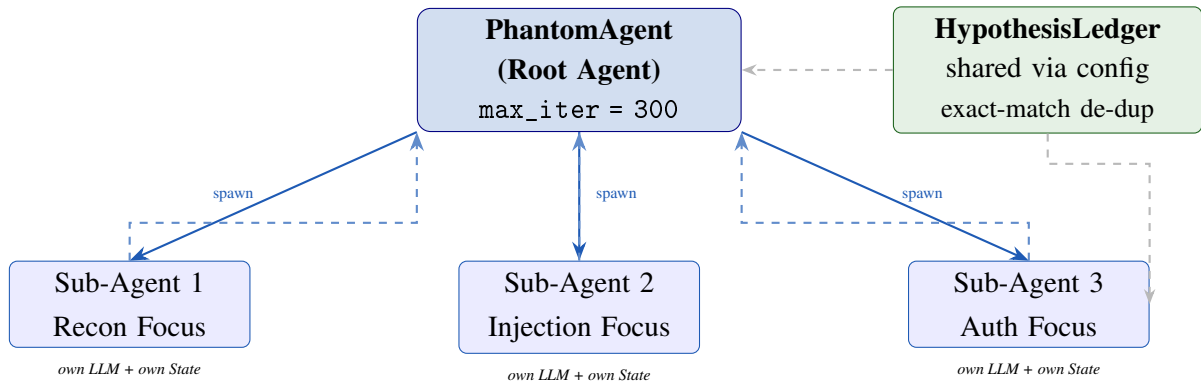


Figure 6.1. Multi-agent runtime topology. Solid arrows = spawn; dashed arrows = result messages via `send_message_to_agent`. Each sub-agent has an independent LLM instance and `AgentState`; the `HypothesisLedger` is shared. Sub-agents do not share `AgentState` with each other or with the root, which means reasoning traces are not shared—only deduplication state.

Architectural limitation. Sub-agents share the `HypothesisLedger` but not their `AgentState`. A finding confirmed by Sub-Agent 1 is added to the shared ledger and to Sub-Agent 1’s `finding_anchors`, but it does not appear in Sub-Agent 2’s message history or anchors unless explicitly communicated via `send_message_to_agent`. Cross-agent finding correlation therefore requires explicit message passing, which may be omitted or delayed.

6.3 ReAct Execution Loop: Implementation

Figure 6.2 shows the complete agent loop as verified in `base_agent.py`, lines 246–486.

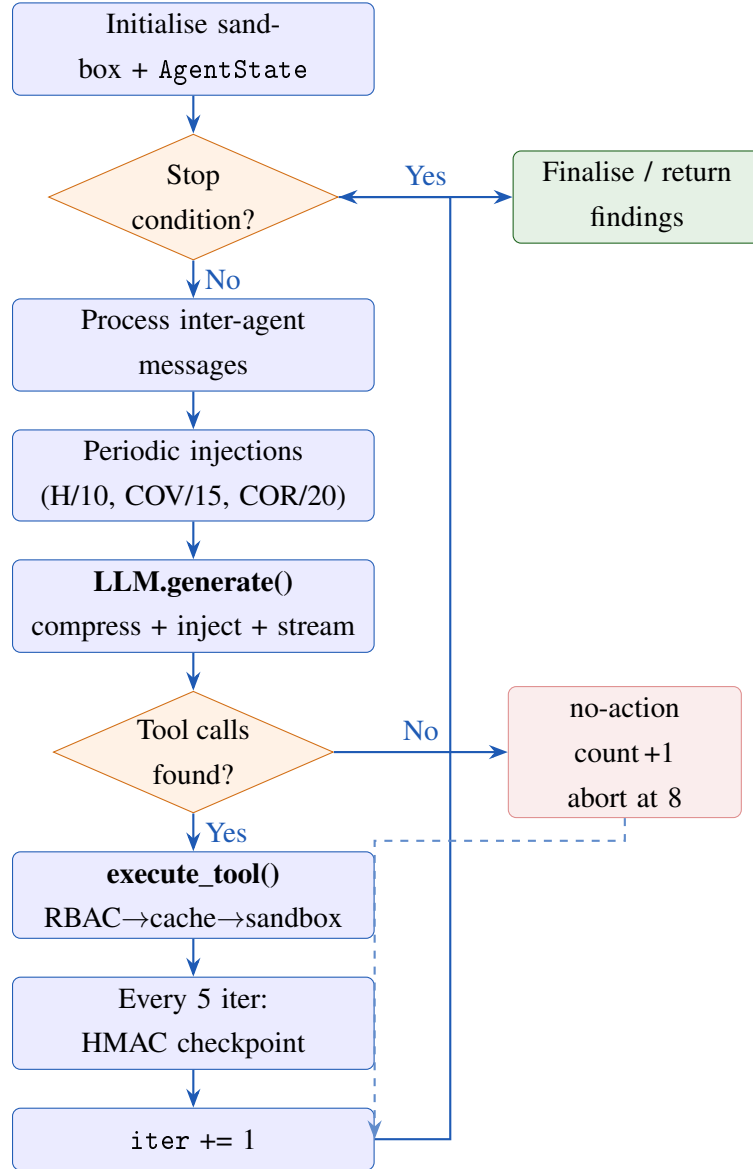


Figure 6.2. *ReAct* execution loop (`base_agent.py:246–486`). Bounded by three termination conditions (Eq. 5.1). Budget-warning messages are injected at 80%, 85%, and $I_{\max} - 3$ iterations.

6.4 LLM Integration and Reliability

6.4.1 Three-State Circuit Breaker

A global circuit breaker singleton (`_CIRCUIT_BREAKER`, `llm.py:181`) aims to reduce cascading failures when the LLM API enters an error state.

The circuit breaker starts in the **CLOSED** state (normal operations). If 5 consecutive retries are exhausted under an error condition, the state transitions to **OPEN**, and a cooldown timer is set. While **OPEN**, all requests are immediately rejected locally. Once the cooldown expires, the breaker enters **HALF_OPEN**, allowing a single test probe. A successful probe resets the state to **CLOSED**; a failure reverts it to **OPEN** with a fresh cooldown timer.

Figure 6.3 shows the state machine diagram.

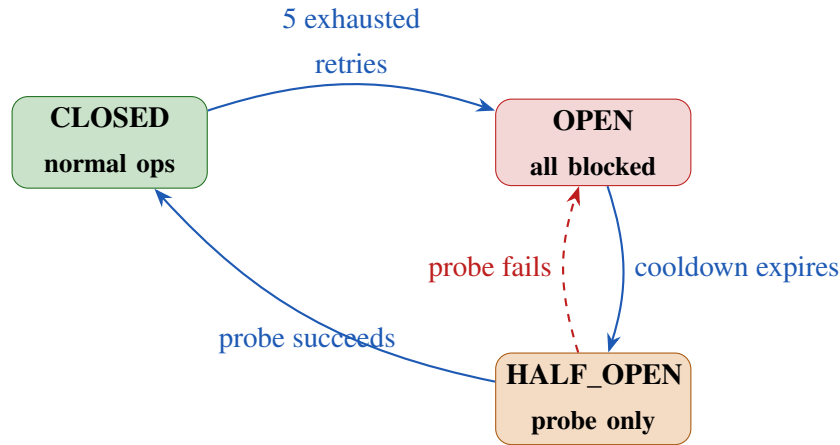


Figure 6.3. Circuit breaker state machine (11m.py:181–250). Transitions are triggered by exhausted retry sequences, not individual HTTP 429 responses. The global singleton means the circuit breaker does not independently track per-agent failure counts.

6.4.2 Graduated Retry Strategy

The `generate()` method implements graduated backoff by failure type:

- **HTTP 429 (rate limit):** exponential backoff $\min(120, 4 \times 2^t)$ s; sets a global `_GLOBAL_RATE_LIMIT_UNT` timestamp shared across all agents to reduce simultaneous retry storms.
- **Context window exceeded:** immediate `_force_compress_messages()` call followed by immediate retry (no sleep).
- **5xx server errors:** $\min(10, 2 \times 2^t)$ s backoff, up to `ratelimit_max_retries` (default 10).
- **All retries exhausted:** attempt fallback LLM; on final failure call `_raise_error()`, which invokes `record_failure()` in the circuit breaker.

6.5 Memory Management

6.5.1 Proactive Compression

Compression is invoked on every LLM call (11m.py:601–610), not only when the context limit is approached. The compressor decides internally whether to compress based on current token count.

The system checks if the token count exceeds 90% of the threshold; if not, the history passes through unmodified. Otherwise, high-signal anchors and recent messages are extracted and preserved verbatim. The remaining older messages are split into blocks of 10 and passed to `_parallel_summarise_chunks()`, an `asyncio` abstraction managed by a bounded concurrency semaphore (limit 4). Finally, the system messages, generated summaries, and recent uncompressed messages are reassembled.

6.5.2 Dual-Anchor Implementation

Figure 6.4 shows the complete memory pipeline. The two anchoring mechanisms are:

1. **Keyword anchors** (message-level, compression-time): messages in M containing keywords in $\mathcal{K}$ are moved to A_m and preserved verbatim from summarisation (llm.py:601–615).
2. **State anchors** (system-prompt-level, every call): `AgentState.finding_anchors` (A_s) are re-injected as a structured user message before every inference call (llm.py:617–642), independently of compression. Even if M is entirely compressed away, A_s remains.

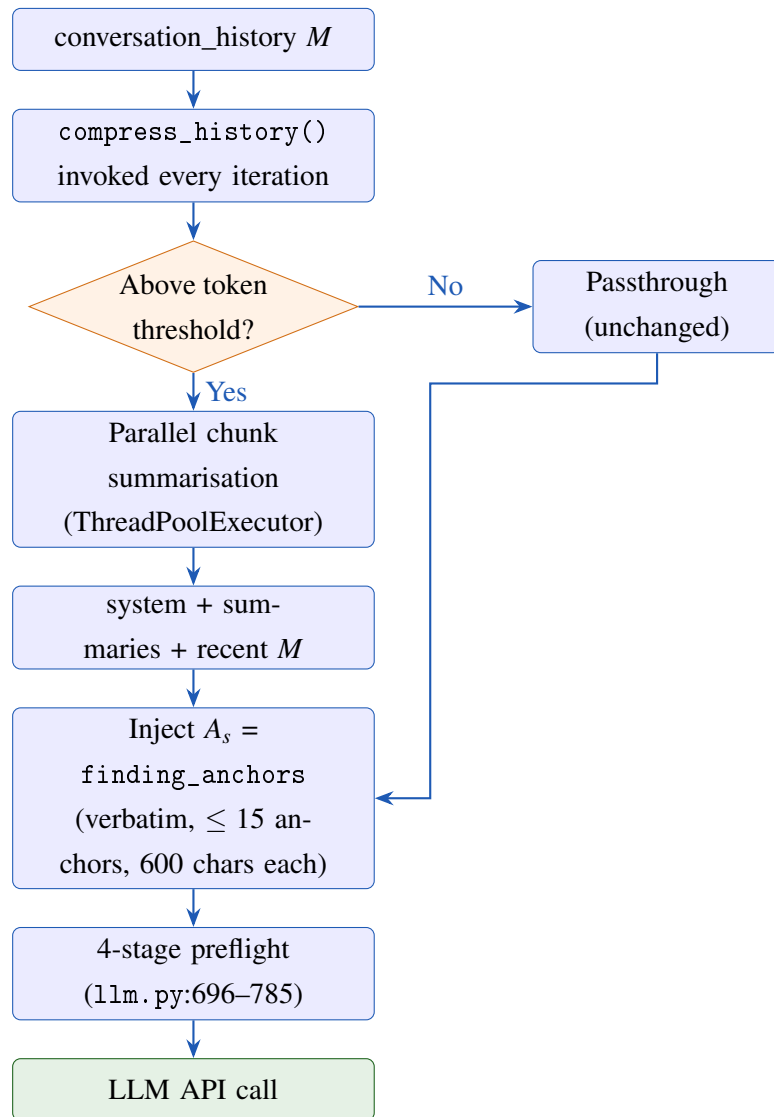


Figure 6.4. Memory management pipeline (memory_compressor.py:598–772, llm.py:601–650). Parallel chunk summarisation uses asyncio with bounded concurrency (semaphore = 4) and falls back to sequential mode if nest_asyncio is unavailable. Dual-anchor invariant Inv-1 holds when both anchoring paths succeed; failure modes documented in Chapter 8.

6.6 Tool Execution Pipeline

Figure 6.5 shows the verified tool execution path in `executor.py`.

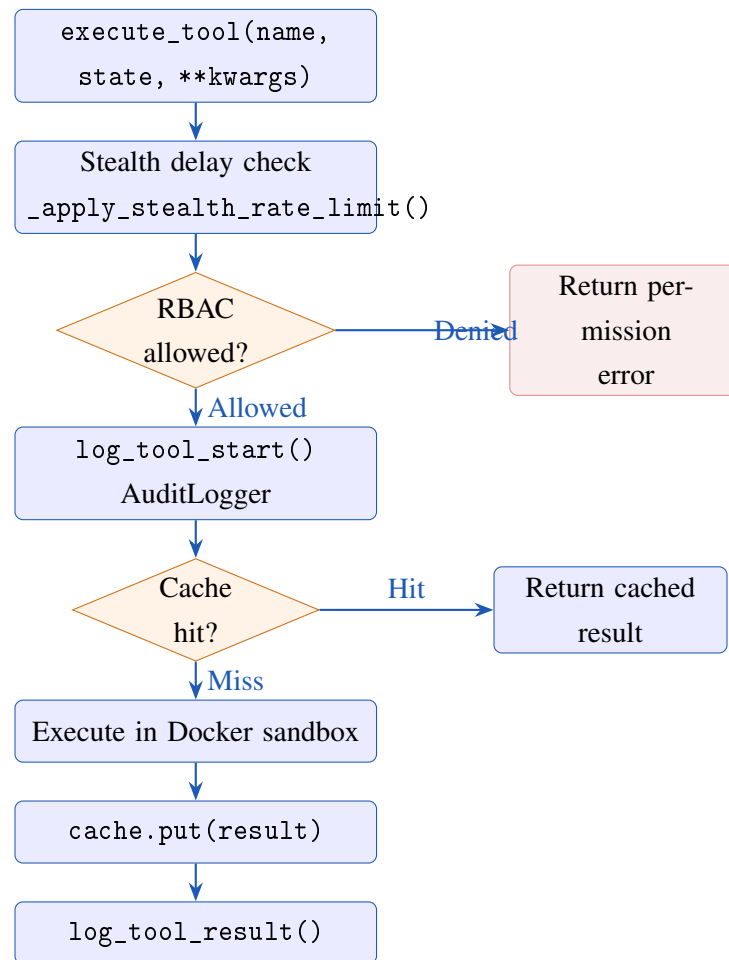


Figure 6.5. Tool execution pipeline (`executor.py`). RBAC uses a lazy import: if the RBAC module is absent, all tools execute without access control (verified: `executor.py`:374–381). The result cache is a top-level import active in all execution paths (`executor.py`:95).

Bug B2 Resolved: Stealth Rate-Limit Now Covers `terminal_execute`

The `_HTTP_TOOLS` frozenset (`executor.py`:34–48) now explicitly registers “`terminal_execute`” with the comment: “*FIX BUG-2: Added ‘terminal_execute’ — was missing from list causing stealth mode bypass*”. Both legacy “`terminal`” and runtime “`terminal_execute`” are present. When `PHANTOM_STEALTH_MODE=true`, all HTTP and terminal tool calls observe the 2.0s minimum inter-call delay. This bug was remediated prior to thesis submission.

6.7 Sandbox and Scope Enforcement

Each scan launches an ephemeral Kali Linux container from `ghcr.io/usta0x001/phantom-sandbox`: (Docker Inc., 2024). The `DockerRuntime` enforces four isolation mechanisms:

1. **Bearer-token authentication:** every tool call to the FastAPI server inside the container must present a per-session token from `AgentState.sandbox_token`.
2. **iptables scope enforcement:** rules generated at container start restrict outbound traffic to registered target CIDRs, aiming to prevent out-of-scope lateral movement.
3. **Execution timeout:** 600 s per tool call.
4. **Orphan cleanup:** `DockerRuntime.__init__()` removes crashed `phantom-scan-*` containers on startup.

6.8 Checkpointing

Every 5 iterations, the agent serialises its state via a five-step atomic write protocol (`checkpoint.py`):

1. Serialise `AgentState` to JSON bytes.
2. Compute HMAC-SHA256 over the bytes.
3. Write to `scan.json.tmp` (temporary file).
4. Atomically rename to `scan.json`.
5. Write HMAC to `scan.json.sig`.

On resume, the HMAC is verified before deserialisation; a tampered or corrupted checkpoint raises `CheckpointIntegrityError`. Maximum work loss is bounded at 5 iterations under this protocol, assuming atomic rename succeeds.

Chapter 7 Design Decisions

This chapter documents the principal design decisions, their considered alternatives, and the engineering rationale for each. These are engineering decisions justified by operational constraints; they are not scientific contributions in themselves.

Table 7.1. Principal Design Decisions with Justification

Decision	Alternative	Rationale
<i>ReAct</i> loop	Hierarchical planner	Self-correcting; failed tool calls are new observations; only structurally complex subtasks require hierarchical planning (Yao et al., 2023)
LiteLLM wrapper	Direct OpenAI SDK	100+ providers; single-variable model switching; scientific reproducibility across models (Berri, 2023)
Flat 31-tool set	Security wrappers	Wrappers failed systematically (v0.9.44 removal); direct capability is simpler and more robust
Pydantic v2 state	Plain dict	Runtime validation; JSON serialisation without custom serialisers
AsyncIO event loop	Threading pool	Python GIL reduces true parallelism; async enables non-blocking sub-agent communication
Docker Kali sandbox	VM / native exec	Disposable; reproducible; enables iptables isolation; Kali pre-installed (Docker Inc., 2024)
Dual-anchor memory	RAG / truncation	Zero latency cost; deterministic; two mechanisms cover each other's failure modes
HMAC checkpoints	No checkpointing	Crash at iteration 200 loses all prior work; 5-iteration granularity limits maximum loss
Exact-match ledger	Embedding-based dedup	Deterministic; reproducible; no vector-store dependency; adequate for same-surface/same-class dedup

7.1 ReAct vs. Hierarchical Planning

Hierarchical task planning (Deng et al., 2024) decomposes a top-level goal into sub-tasks before execution begins. This approach requires an accurate model of the target before any tool is executed, which is unavailable in black-box penetration testing. *ReAct*'s iterative model—where each action is selected based on actual observations, including failure observations—is more appropriate for partially observable, open-ended target spaces.

7.2 Exact-Match vs. Semantic Deduplication

Vector-store retrieval-augmented generation would enable semantic similarity matching for hypothesis deduplication. The operational requirement, however, is to avoid re-testing the same (surface, vulnerability class) pair within a scan session. Exact-match membership testing on string equality is deterministic, reproducible, and has zero infrastructure overhead compared to embedding-based retrieval. The limitation—near-duplicate hypotheses are not caught—is documented in Chapter 8 and is accepted as a design trade-off in favour of simplicity and reproducibility.

7.3 Dual Anchoring vs. Single-Mechanism Approaches

Uniform summarisation (LangChain's `ConversationSummaryMemory`) treats all messages as equally expendable. Sliding window approaches preserve recency but lose early findings. The dual-anchor design preserves a specific, content-identified subset of messages (keyword-matching findings) while summarising the remainder, and supplements this with state-level injection that is independent of compression. T-4 (Section 4.2) argues that both mechanisms are required because they fail in complementary scenarios: keyword anchoring fails when keywords are absent; state anchoring fails when the state-update logic does not run.

Chapter 8 System Limitations and Failure Modes

Every mechanism described in this thesis has identifiable failure modes. This chapter documents them systematically. The purpose is not to undermine the contributions but to bound the claims precisely and provide a research agenda for future mitigation.

Failure Mode F2 – Compression-Induced Semantic Drift

LLM summarisation of compressible message chunks may alter the semantic content of the original messages. A probe result stating “SQL injection confirmed via UNION-based extraction on `/api/search?q=`” may be summarised as “injection vulnerability found”, losing the specific endpoint, technique, and parameter.

Impact: The agent’s subsequent reasoning operates on a degraded representation of earlier findings, potentially leading to incorrect hypothesis generation or missed follow-up probes.

No complete mitigation without verbatim preservation of all high-signal messages, which conflicts with the compression objective.

Failure Mode F3 – Near-Duplicate Hypotheses

The exact-match deduplication rule does not handle semantic near-duplicates. Hypotheses with identical meaning but different string representations—“`/api/users, IDOR`” vs. “`/api/users endpoint, Insecure Direct Object Reference`”—are treated as distinct entries and will both be tested.

Impact: Redundant testing of the same attack surface, wasting computational budget and potentially triggering WAF or rate-limit detection on the target.

Mitigation path: Embedding-based similarity search on hypothesis strings, not currently implemented.

Failure Mode F4 – Agent Reasoning Inconsistency

Sub-agents share the `HypothesisLedger` but not their `AgentState`. A finding confirmed by sub-agent *i* is visible to sub-agent *j* only via dedicated `send_message_to_agent` calls. If these calls are omitted or delayed, sub-agent *j* may generate hypotheses that contradict confirmed findings from sub-agent *i*.

Impact: Inconsistent reasoning state across parallel agents; potential retesting of proven-vulnerable surfaces with different payloads.

Partial mitigation: The shared `HypothesisLedger` prevents re-testing the exact same (surface, class) pair. It does not prevent different payloads on the same pair.

Failure Mode F5 – Tool Hallucination Mismatch

The agent’s reasoning trace may assert tool execution results that differ from actual tool outputs. This occurs when the LLM generates a plausible-sounding Observation in its thought trace without verifying against the actual tool response.

Impact: Downstream hypotheses are grounded in fabricated evidence, leading to incorrect vulnerability assessments.

Partial mitigation: The *ReAct* design requires all tool results to come from actual execution; the agent’s reasoning trace is not used as ground truth. However, the agent may quote or misquote tool outputs in subsequent reasoning, and this is not verified programmatically.

Chapter 9 Discussion

9.1 Scientific Value of the Contributions

The four contributions (C1–C4) differ in nature and defensibility:

C1 (CVE Template Gap) provides a conceptual result: a named, formally defined structural property of template-dependent scanners. The value is principally terminological and methodological—it gives practitioners a precise name and characterisation for a failure mode they have observed empirically. The formal definition is straightforward; the contribution lies in making it explicit.

C2 (Dual-Anchor Memory) is an architectural design with a stated invariant. The invariant is not proven correct under all conditions; its failure modes are documented (F1–F2). The contribution is the design pattern: distinguishing expendable from irreplaceable messages and applying two independent preservation mechanisms. Whether this design is empirically adequate at maintaining finding preservation across real scans requires controlled evaluation (Section 9.3).

C3 (Deterministic Deduplication) is the most operationally bounded contribution: exact-match membership testing is deterministic, reproducible, and implementable without external dependencies. Its limitation (F3) is clear and does not undermine the core claim that it reduces redundant testing for exact surface–class pairs.

C4 (Code-Verified Methodology) is a methodological contribution with concrete output: the verification matrix (Appendix ??) that binds 17 architectural claims to source locations, identifies four dead code components, and documents two bugs that were subsequently remediated. Its value generalises beyond *Phantom*: any LLM-agent paper that applies this methodology produces more falsifiable and trustworthy claims.

9.2 Positioning Against Prior Work

Phantom addresses combinations of properties that individual prior systems do not. Pen-testGPT addresses LLM-based security reasoning but not context management. LangChain agents address context management but not security-specific hypothesis tracking or crash resilience. No prior open-source system combines dual-anchor memory, deterministic hypothesis deduplication, HMAC checkpointing, and multi-agent coordination in a security testing context.

This positioning does not imply superiority in detection quality or coverage. Without empirical evaluation, the claim is architectural: *Phantom* instantiates a set of mechanisms

that, as a combination, has not been implemented in prior open-source systems.

9.3 Claims That Require Empirical Validation

The following claims are stated as design theses but are not empirically supported in this thesis. Each requires a controlled experiment for validation:

1. **RQ2 validation:** The dual-anchor mechanism is empirically adequate at preserving critical findings across compression. A controlled evaluation would inject synthetic high-signal messages, apply compression, and measure recall of specific findings.
2. **RQ3 validation:** Exact-match deduplication reduces redundant tool calls in practice. Measurement: count duplicate (surface, class) pairs in baseline (no ledger) vs. ledger-equipped runs across matched scan sessions.
3. **Detection capability:** Whether a *ReAct*-based agent achieves higher recall on intentionally vulnerable targets than template-based scanners. Evaluation: paired runs on standardised targets (Juice Shop, DVWA, WebGoat) with ground-truth vulnerability lists (Appendix ??).
4. **F5 frequency:** How often tool hallucination mismatch affects downstream reasoning in practice. Measurement: systematic comparison of agent-quoted tool outputs vs. logged actual outputs.

These evaluations are beyond the scope of this thesis. They constitute the primary empirical agenda for future work.

9.4 Failure Mode Prioritisation

F6 (RBAC bypass) and F5 (hallucination mismatch) are the highest-severity failure modes. F6 is a security property failure: the system may execute tools without intended access control, which is a correctness issue with security implications in any deployment. F5 is a reasoning correctness issue: if the agent grounds downstream decisions in fabricated evidence, all subsequent findings may be unreliable. Both should be addressed before any production deployment of the system.

F1–F4 are architectural limitations with partial mitigations. They represent design trade-offs rather than correctness failures.

Conclusion

Summary

This thesis presented *Phantom*, a ReAct-based multi-agent autonomous penetration testing system, as the vehicle for four bounded architectural contributions.

C1 provides a formal, named description of the CVE Template Gap—the structural detection ceiling of template-dependent scanners—establishing that orchestration improvements cannot reduce this gap and that evaluation studies using training targets must account for it when selecting baselines.

C2 proposes the dual-anchor memory architecture, a design that aims to preserve high-signal security findings across context compression by combining message-level keyword anchoring with state-level anchor injection. The invariant $A \subseteq C(M) \cup A_s$ holds when both preservation mechanisms succeed; its failure modes are documented.

C3 defines a deterministic, exact-match hypothesis deduplication mechanism that reduces redundant testing for identical (surface, vulnerability class) pairs without requiring vector-store infrastructure. Its limitation to exact-match equality is accepted as a design trade-off.

C4 instantiates a code-verified architectural methodology, producing a source-traceable claim map (Appendix ??) that identifies four unreachable dead code components, two operational bugs subsequently remediated in the codebase (B1, B2), one live security risk (F6), and one module (EnhancedAgentState) correctly reclassified from *dead code* to *conditionally active checkpoint/resume mechanism*.

No empirical evaluation results are reported. Section 9.3 proposes a controlled evaluation methodology for each untested claim.

Theoretical Perspectives

Several theoretical directions follow from this analysis:

1. **Formal CVE Template Gap characterisation:** the gap is currently defined set-theoretically. A complexity-theoretic characterisation—relating exploitability to template library coverage—could provide a quantitative measure of gap size as a function of target novelty.
2. **Memory-optimal compression policies:** an information-theoretic relevance scoring function for messages could optimise the trade-off between compression ratio and finding preservation, replacing the current heuristic keyword anchoring with a principled assignment of preservation priority.

3. **Semantic hypothesis deduplication:** embedding-based similarity search on hypothesis descriptions could reduce redundant testing for near-duplicate surface–class pairs not caught by exact matching, at the cost of increased infrastructure complexity.
4. **Model-independent evaluation:** the five design theses describe properties that should hold across LLM providers. A multi-model comparative study using the provider-agnostic LiteLLM interface could isolate the contribution of reasoning quality from architectural design, providing a foundation for more general claims.
5. **Adaptive iteration budget:** the current loop uses a fixed iteration budget. An adaptive policy allocating budget proportionally to discovered attack surface density could improve coverage efficiency on heterogeneous targets.

References

- Berri, K. (2023). *LiteLLM: Call all LLM APIs using the OpenAI format*. <https://github.com/BerriAI/litellm>
- Bjoern, K. (2024). *OWASP Juice Shop: Probably the most modern and sophisticated insecure web application*. <https://owasp.org/www-project-juice-shop>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Chase, H. (2023). *LangChain: Building applications with LLMs through composability*. <https://github.com/langchain-ai/langchain>
- Deng, G., Liu, Y., Mayoral-Vilches, V., Liu, P., Li, Y., Xu, Y., Zhang, T., Liu, Y., Pinzger, M., & Rass, S. (2024). PentestGPT: Evaluating and harnessing large language models for automated penetration testing. *33rd USENIX Security Symposium*.
- Docker Inc. (2024). *Docker: Accelerated container application development*. <https://www.docker.com>
- Happe, A., & Eckmaier, J. (2023). Getting pwn'd by AI: Penetration testing with large language models. *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.
- ISC2. (2023). *ISC2 Cybersecurity Workforce Study 2023* (tech. rep.). International Information System Security Certification Consortium. <https://www.isc2.org/research/workforce-study>
- OWASP Foundation. (2021). *OWASP Top 10: 2021 edition*. <https://owasp.org/Top10>
- OWASP Foundation. (2024a). *OWASP WebGoat: A deliberately insecure application*. <https://owasp.org/www-project-webgoat>
- OWASP Foundation. (2024b). *OWASP ZAP: The Zed Attack Proxy*. <https://www.zaproxy.org>
- Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*.
- ProjectDiscovery. (2024). *Nuclei: Fast and customizable vulnerability scanner*. <https://github.com/projectdiscovery/nuclei>
- Russell, R. L., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., Ellingwood, P., & McConley, M. (2018). Automated vulnerability detection in source code using deep representation learning. *17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 757–762.

- Sutton, M., Greene, A., & Amini, P. (2007). *Fuzzing: Brute force vulnerability discovery*. Addison-Wesley.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824–24837.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*. <https://arxiv.org/abs/2308.08155>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2210.03629>